

Simulacro Primer Parcial

Primer Semestre - 29 de abril de 2026
Parte 2 - Individual - Implementación en Java

Importante

- Duración: 1:00 hs.
 - Material permitido: material propio.
 - No puede usar asistentes de programación o consultar a IAs.
 - No editar las clases e interfaces provistas bajo el package *library*
-

Escenario

Año 2147. Las misiones de exploración en el espacio profundo se han vuelto cada vez más complejas. Las naves autónomas deben operar durante largos períodos sin intervención humana directa, enfrentando fallas técnicas, eventos inesperados y una alta demanda de procesamiento de tareas simultáneas.

Sin embargo, varios incidentes recientes han evidenciado problemas en la forma en que estas naves gestionan sus tareas operativas: saturación del sistema, demoras en la ejecución de tareas críticas y dificultades para acceder rápidamente al historial de operaciones.

Ante esta situación, distintas agencias espaciales deciden contratar a su empresa unipersonal AEP (Aprobando Este Parcial) para diseñar un nuevo sistema de gestión de tareas que permita mejorar la eficiencia y confiabilidad de las operaciones.

El sistema deberá encargarse de administrar las tareas que recibe la nave en tiempo real. Cada tarea posee la siguiente información:

- ID único
- Descripción
- Nivel de criticidad (de mayor a menor: 1, 2, 3 y 4)

Requerimientos del sistema

R1. Recepción de tareas

El sistema recibe tareas de forma continua.

- Las tareas de criticidad 1 y 2 tienen prioridad sobre las demás.
- Las tareas de criticidad 3 y 4 deben organizarse respetando estrictamente el orden en que fueron recibidas.
- El sistema puede almacenar como máximo 25 tareas pendientes en total.
- Adicionalmente, dentro de las tareas pendientes no puede haber más de 10 tareas de criticidad 1 o 2 simultáneamente.

- Por lo tanto:
 1. Si el sistema alcanzó el máximo de tareas pendientes, no se aceptan nuevas tareas hasta liberar alguna.
 2. Si ya existen 10 tareas críticas pendientes, una nueva tarea de criticidad 1 o 2 no se incorpora al sistema hasta que la cantidad de tareas críticas descienda al menos a 9.

R2. Procesamiento de tareas

El sistema ejecuta las tareas siguiendo las siguientes reglas:

- Siempre se selecciona para ejecutar primero la tarea de mayor criticidad disponible.
- En caso de empate en el nivel de criticidad, se ejecuta primero la tarea que haya llegado antes.

R3. Registro de tareas procesadas

- El sistema mantiene un historial de tareas ejecutadas.
- Una vez que se hayan procesado al menos 75 tareas, estas deberán almacenarse en una estructura que permita:
 1. Buscar una tarea a partir de su ID.
 2. Garantizar tiempo de búsqueda logarítmico en todos los casos, independientemente del orden de inserción.

R4. Cancelación de tareas

- El sistema debe permitir cancelar una tarea pendiente a partir de su ID.
- Solo pueden cancelarse tareas que aún no hayan sido procesadas.

Nota: Los métodos entregados de forma obligatoria en la implementación de los TDAs están disponibles para su uso y no deben escribirse nuevamente en pseudocódigo.

Tareas

1. Descargar de la web [signature](#) el archivo “2026S1-parcial-1-parte2.zip” que contiene el Proyecto Maven a ser completado y un archivo “naves.txt”
2. Implementar las clases necesarias para representar las tareas y el sistema de gestión de la nave, incluyendo atributos y métodos relevantes.
3. Implementar un método `recibirTarea` que permita incorporar nuevas tareas al sistema respetando las restricciones del requerimiento **R1**, a partir de la lista incluida en “naves.txt”. Recuerde que, en caso de no poder incorporarse inmediatamente por restricciones de capacidad o criticidad, la tarea deberá mantenerse en espera hasta que pueda ser ingresada al sistema.
4. Implemente un método `procesarTarea` que seleccione y ejecute la próxima tarea según las reglas establecidas en el requerimiento **R2**. Recuerde que luego de ejecutarse, la tarea debe almacenarse en el historial de tareas procesadas.
5. Implemente un método `buscarTareaProcesada(id)` que permita recuperar una tarea ejecutada a partir de su ID.
6. Implemente un método `CancelarTarea(id)` que permita eliminar una tarea antes de ser procesada a partir de su ID.

Entregable

1. Desarrollo de la funcionalidad especificada (50 %).
2. Desarrollo del programa ejecutable **Main** con los datos de ejemplo provistos (20 %).
3. Desarrollo de los casos de prueba para las funcionalidades requeridas (30 %).

Se debe entregar **todo** el Proyecto Maven en la tarea correspondiente de la webasignatura.